Processos de Software

Backlog e Programação Extrema (XP)

DIM0510 — Turma 01 · Aulas 03–04 (Sprint 0) · 31/08 e 02/09

Prof. Fernando · UFRN · 2026.2

Roteiro da semana

Segunda, 31/08 — Backlog na prática

Bloco	O que vemos
A arquitetura molda o backlog	Componentes, e por que o backlog é assim
Fatiar vertical	A habilidade que separa história de tarefa
Padrões de divisão	Como quebrar uma história grande demais
Mapa de história	Enxergar o produto, e o MVP dentro dele
Estimar e "pronto"	Régua, velocidade, critérios e a DoD

Quarta, 02/09 — Extreme Programming

Bloco	O que vemos
Valores e práticas	O "como" que o Scrum deixa em aberto
TDD e refatoração	Design que evolui, com rede de segurança
Pair e revisão	Propriedade coletiva do código
No MUSI	Onde cada prática deixa evidência

Onde paramos

Na aula passada, o Scrum e uma primeira passada pelo backlog, para viabilizar a simulação de Planning:

```
História de usuário: Como… Quero… Para…
INVEST como teste de qualidade
Estimativa em pontos, planning poker, velocidade
Definição de Pronto
```

Aquilo bastou para planejar uma sprint. Hoje aprofundamos os pontos que mais custam na prática, e aplicamos ao backlog do projeto.

Onde estudar depois

Fonte	Foco
Rubin — *Essential Scrum*, cap. 6	Backlog, refinamento e estimativa

Disponível no SIGAA.

A arquitetura molda o backlog

Antes das histórias, entender o produto.

Todo produto é feito de componentes que conversam

Mesmo o app mais simples tem partes com papéis distintos, que trocam mensagens:

```
Cliente / tela  ──►  API / serviço  ──►  dados
   (o que o           (as regras          (onde
    usuário vê)        e a orquestração)   persiste)
```

No MUSI, isso é concreto: um app, uma API (Ktor ou Quarkus), um serviço de busca em Go, e o contrato compartilhado que todos seguem.

> Vocês não precisam saber programar as três partes para escrever boas histórias. Precisam entender que elas existem e como se comunicam — é isso que torna uma história "demonstrável de ponta a ponta".

Por que a arquitetura decide a forma do backlog

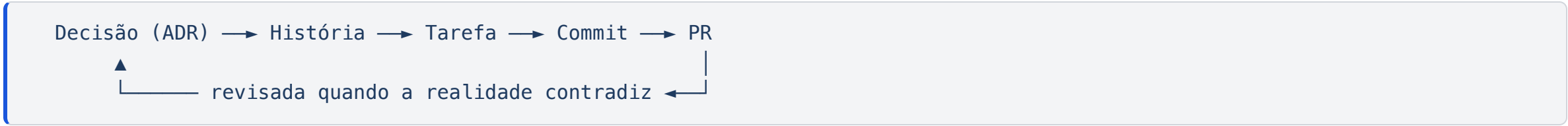
Entender os componentes muda como se escreve e organiza o backlog:

A arquitetura tem…	…e o backlog ganha
Componentes distintos	O campo `Componente` no quadro (docs, api, serviços, app)
Comunicação entre eles	Histórias que atravessam mais de um componente
Decisões de projeto	Itens do tipo `Decisão` (uma ADR), antes da história
Partes que só o time entende	Itens de `Débito` e de risco, explícitos

> Uma "fatia vertical" é justamente uma história que corta esses componentes de cima a baixo. Sem enxergar a arquitetura, é fácil escrever histórias que param numa camada só — e não entregam nada.

A cadeia que liga arquitetura e backlog

No MUSI, cada peça do processo se conecta à anterior:



- Uma decisão de arquitetura (ex.: "a conciliação vai para um serviço Go") vira uma ADR
- A ADR habilita histórias; a história se quebra em tarefas; a tarefa vira commit e PR
- Pensar assim é pensar de forma sistêmica: cada item sabe de onde veio e por quê

> `processo/README.md` do MUSI mostra essa cadeia funcionando num projeto real.

Foco na entrega de valor

```
Backlog "de tarefas"          Backlog de valor
──────────────────────        ────────────────
Criar o banco de dados        Registrar uma despesa
Configurar autenticação       Ver quem deve a quem
Fazer a tela de login         Entrar para ver meu grupo
Modelar as entidades          Fechar a conta do mês
```

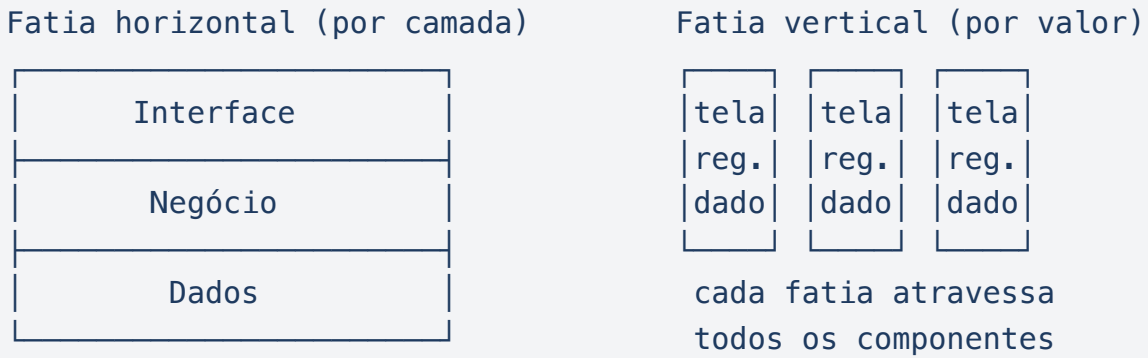
A coluna da esquerda é trabalho; a da direita é resultado para alguém. O backlog contém itens que entregam um valor tangível.

> Se ninguém percebe nada quando o item termina, ele não é uma história — é uma tarefa. Tarefa vive dentro da história, não no topo do backlog.

Parte 1

Fatiar vertical

Horizontal versus vertical



A fatia horizontal só entrega valor quando a última camada fica pronta. A vertical entrega uma função inteira, estreita, desde já — cortando os componentes que acabamos de ver.

> "Fazer todo o banco" é fatia horizontal: não dá para demonstrar nada até o fim. Fatie por função, mesmo que a função seja mínima.

Uma história vertical, por menor que seja

```
Como membro do grupo
Quero registrar uma despesa com valor e quem pagou
Para que ela entre na conta compartilhada
```

Essa fatia atravessa tudo: tem tela, tem regra, toca os dados. Estreita, mas inteira.

Sinal de fatia vertical	Sinal de fatia horizontal
Demonstrável ao fim da história	Só demonstra quando outra história terminar
O usuário percebe a diferença	"Preparei a base para depois"
Cabe numa sprint	Cresce sem fim ("todo o CRUD")

> A pergunta-teste: *dá para mostrar isso funcionando na Review?* Se a resposta depende de outra história, a fatia está horizontal.

Parte 2

Padrões de divisão

Quando a história não cabe numa sprint.

SPIDR — cinco formas de quebrar

Letra	Divida por	Exemplo (despesa compartilhada)
Spikes	Investigação primeiro	Testar duas libs de gráfico antes de decidir
Paths	Caminhos do fluxo	Pagar por Pix / por cartão / em dinheiro
Interfaces	Plataforma ou canal	Primeiro no navegador; depois no celular
Data	Fatia dos dados	Só despesas em reais; outras moedas depois
Rules	Regras de negócio	Divisão igual agora; por porcentagem depois

Mike Cohn, *SPIDR*.

> Não precisa decorar os cinco. Diante de uma história grande, tente um deles em vez de empurrar a história inteira para a sprint.

Caminho feliz primeiro

A divisão mais barata: separe o caso comum dos casos de borda.

```
História grande:  Registrar despesa
     │
     ├── (1) Valor válido, um pagador           ← caminho feliz
     ├── (2) Valor inválido é recusado          ← borda
     ├── (3) Dividir entre vários participantes ← extensão
     └── (4) Editar uma despesa já lançada      ← extensão
```

> Entregue o item (1) primeiro: é a fatia que prova a ideia. As bordas viram histórias próprias, priorizadas por valor.

Sinais de que precisa dividir

Sinal	O que costuma esconder
A estimativa de esforço está enorme	Pode ser dividido em itens menos complexos
O "e" no título (*registrar e editar*)	Duas histórias grudadas
Ninguém consegue estimar	Ainda não foi entendida
Muitos critérios de aceitação	Vários resultados diferentes juntos

> A conjunção "e" no título de uma história é quase sempre a linha pontilhada onde ela se divide em duas.

Parte 3

Mapa de história

Do backlog em lista ao produto inteiro.

O problema da lista plana

Um backlog é uma lista ordenada — ótima para saber o próximo item, ruim para enxergar o produto como um todo. É fácil priorizar bem itens e ainda assim montar um MVP que não funciona de ponta a ponta.

```
Lista:   [ item, item, item, item, item, ... ]
         você vê a ordem, não vê a jornada
```

O mapa de história (Jeff Patton) organiza o backlog em duas dimensões: a jornada do usuário na horizontal, o detalhe na vertical.

A forma do mapa

```
Atividades →   Entrar          Registrar despesa     Fechar o mês
               ————————        ——————————————————    ————————————

Passos →       criar grupo     lançar valor          ver saldos
               entrar          escolher pagador      marcar pago
——————————————————————————————————————————————————————————————  ← linha do MVP
Depois →       convite         dividir por %         exportar PDF
               foto perfil      editar despesa        gráfico
```

- No topo, a espinha: as atividades, na ordem em que o usuário as faz
- Abaixo, as histórias que realizam cada passo, mais detalhadas embaixo
- A linha é o corte do MVP: o mínimo que atravessa a jornada inteira

> O MVP é uma fatia horizontal do mapa: um pouco de cada atividade, ponta a ponta, e não uma atividade inteira e nenhuma das outras.

Por que o mapa ajuda o MVP

Sem mapa	Com mapa
MVP vira "as histórias do topo da lista"	MVP atravessa a jornada inteira
Descobre-se um passo faltando na Review	O buraco na espinha aparece antes
"Está 80% pronto" (mas não usa)	Cada corte é utilizável de ponta a ponta

> Um MVP que faz login e não deixa registrar nada está "80% pronto" e serve a ninguém. O mapa expõe isso na Sprint 0, não mais pra frente no projeto.

Parte 4

Estimar e "pronto"

Calibrar, prever, e definir o que termina.

Ponto é relativo, a uma referência

Pontos não têm significado sozinhos. Ganham sentido contra uma história de referência que a equipe conhece:

```
Escolham juntos:
  uma história pequena e clara  →  2 pontos   (a "régua")

Estimem o resto por comparação:
  "isto é o dobro daquela"      →  4
  "isto é bem menor"            →  1
```

Sem uma régua compartilhada, cada pessoa estima numa escala própria. Definam a história de 2 pontos -- ou ainda selecionem uma referência para história P, outra para M e outra para G, se estiverem usando a metáfora do tamanho de roupa -- antes de estimar o backlog.

Triangulação

Confira uma estimativa contra duas outras já feitas, não contra horas:

```
Esta história de 5   deve ser...
   maior   que aquela de 3    ✓
   menor   que aquela de 8    ✓
```

Se um "5" parece maior que um "8" já estimado, algo está errado em um dos dois — e essa conversa vale mais que o número final.

> Estimativa boa é consistente, não precisa. O objetivo é que 5 seja sempre maior que 3, não que 5 de um valor objetivo (em horas, por exemplo). O mesmo se aplica para a metáfora do P, M e G.

Velocidade e previsão

Velocidade é quantos pontos a equipe conclui por sprint. Serve para prever, e só a partir da terceira sprint, quando há média:



Use velocidade para	Nunca use para
Prever quantas sprints faltam	Comparar grupos
A própria equipe se planejar	Meta imposta de fora

> Transformar velocidade em meta faz a equipe inflar estimativas, e a métrica morre. Ela é um instrumento de previsão, não de cobrança.

Critério de aceitação em Dado/Quando/Então

O formato Gherkin torna o critério verificável, e é o rascunho direto do teste:

```
História: registrar uma despesa compartilhada

  Dado    que estou num grupo com 3 pessoas
  Quando  registro uma despesa de R$ 90 paga por mim
  Então   o saldo mostra que cada um me deve R$ 30

  Dado    um valor zero ou negativo
  Quando  tento salvar
  Então   a despesa é recusada, com mensagem
```

O "Dado" é o estado inicial; o "Quando" é a ação; o "Então" é o resultado observável.

> Se você não consegue escrever o "Então", também não vai conseguir escrever o teste.

Pronto não é o mesmo que aceito

	Aplica-se a	Definido por
Definição de Pronto	Todos os itens	A equipe
Critérios de aceitação	Aquele item	O Product Owner

Um item está completo quando cumpre os dois. Ele pode passar em todos os seus critérios e ainda não estar pronto — por não ter sido revisado, ou por quebrar a CI.

> A Definição de Pronto também evolui: comece pelo que dá para cumprir hoje (na branch, revisado em PR, critérios conferidos) e evolua os critérios a cada sprint (testes, CI passando).

Parte 5

Um exemplo de backlog

O backlog do MUSI, no GitHub

O projeto de referência tem um quadro configurado, aberto para vocês copiarem:

```
github.com/users/fmarquesfilho/projects   →   Project "MUSI"
```

Campo	Valores	Por que existe
`Status`	A fazer · Em andamento · Em revisão · Pronto	O fluxo
`Tipo`	História · Tarefa · Defeito · Decisão · Débito	Muda a definição de pronto
`Componente`	docs · contratos · api · services · app	Liga o item à arquitetura
`Tamanho`	P · M · G	Relativo, nunca em horas
`Risco`	Baixo · Alto	O primeiro item puxado na sprint

> O passo a passo para montar o seu está em `processo/GITHUB-PROJECTS-SETUP.md`, e a explicação de cada campo em `processo/backlog.md`.

Poucos campos, cada um com um porquê

O quadro do MUSI é deliberadamente enxuto: quadros com muitos campos ficam desatualizados, porque preenchê-los deixa de compensar.

- `Componente` é o que liga cada item à arquitetura que vimos no começo da aula
- `Risco: Alto` significa que ainda não se sabe se a abordagem funciona — e a regra é puxar esse item primeiro, para falhar cedo
- `Tamanho: G` avisa que o item ainda precisa ser fatiado, e por isso não entra em sprint

> Um item de risco alto entrega principalmente informação, mesmo quando o resultado técnico não é o esperado. Por isso ele tem o mesmo peso na avaliação.

Parte 6

Oficina e próximos passos

Exercício

Com o backlog inicial do grupo aberto no GitHub Projects:

1. Achem a maior história. Fatiem-na com um padrão do SPIDR
2. Achem uma tarefa disfarçada de história. Reescrevam como valor,
   ou marquem como tarefa de outra história
3. Rascunhem o mapa: 3 a 5 atividades no topo, e a linha do MVP
4. Escolham a história de referência de 2 pontos
5. Estimem 5 histórias por comparação a uma ou mais referências já estimadas
6. Escrevam os critérios da história do topo em Dado/Quando/Então

No fim, cada grupo mostra a história que mais mudou depois de fatiada.

Quarta · 02/09

Extreme Programming

Onde o Scrum para, o XP começa

O Scrum organiza quando as decisões acontecem e quem as toma. Não diz como programar.

```
Scrum   quando · quem · o quê    cadência, papéis, backlog
XP      como                     testes, refatoração, pares, integração
```

Extreme Programming (Kent Beck, 1999) é o conjunto de práticas técnicas que sustenta a entrega frequente.

> Equipe com Scrum organizado e sem prática técnica entrega rápido por duas sprints e depois afunda em dívida. As práticas de fundamentos conceituais do XP ajudam a manter um "ritmo sustentável".

XP: levar o feedback ao extremo

Pegar o que sabidamente funciona e fazer o tempo todo:

Se revisar código é bom…	…revise o tempo todo (pair programming)
Se testar é bom…	…teste antes de escrever (TDD)
Se integrar é bom…	…integre várias vezes ao dia (CI)
Se design simples é bom…	…refatore para mantê-lo simples

> "Extremo" não é atitude: é encurtar cada ciclo de feedback até ele caber em minutos.

Os cinco valores

Valor	O que orienta
Comunicação	O código e os testes conversam pela equipe
Simplicidade	Faça a coisa mais simples que funcione hoje
Feedback	Do teste, do par, da CI, do usuário — quanto antes
Coragem	Refatorar, apagar código, dizer "não está pronto"
Respeito	Ninguém quebra o build dos outros de propósito

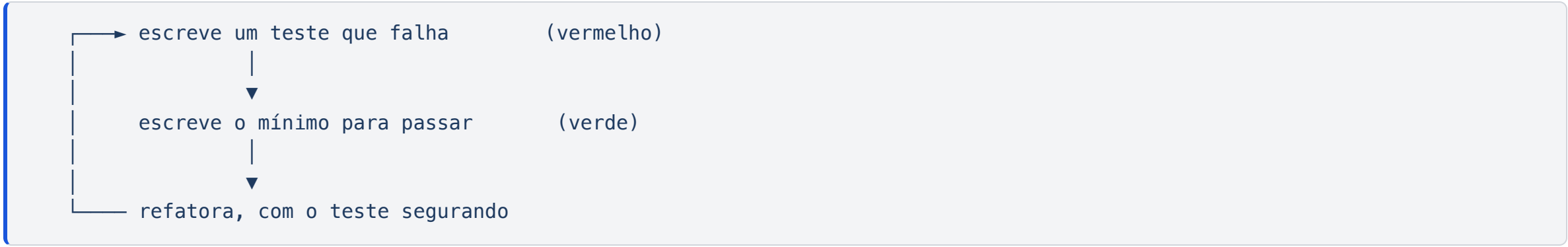
> Cada prática do XP é uma forma concreta de viver um destes valores.

As práticas, em três anéis

```
Feedback fino      TDD · pair programming · equipe junta
Processo contínuo  integração contínua · refatoração · releases pequenas
Entendimento       design simples · propriedade coletiva · padrão de código
```

Nenhuma se sustenta sozinha: TDD sem CI acumula teste que ninguém roda; refatoração sem teste é arriscada.

TDD: o ciclo



O teste vem antes e define o que "pronto" significa para aquele pedaço.

> O ganho não é só o teste no fim: é o design. Código difícil de testar costuma ser código mal desenhado — o TDD traz esse retorno em minutos, não em semanas.

Design evolutivo e refatoração

Você não precisa acertar o design de primeira. Precisa manter o código simples e mudá-lo com segurança.

Prática	O que evita
Design simples	Abstração especulativa que nunca é usada
Refatoração contínua	Evita refatorações gigantes
Rede de testes	Medo de mexer no que funciona

No MUSI, cada decisão de design que não é óbvia vira uma ADR — e uma ADR pode ser substituída quando a realidade a contradiz.

> `docs/decisoes/`. O design não é congelado no início: é registrado e revisado. Refatorar fica barato quando os testes cobrem as funcionalidades principais e a decisão está documentada.

A regra de ouro: coesão e acoplamento

No fundo, a qualidade de um projeto cabe numa frase: cada parte faz uma coisa (coesão alta) e sabe o mínimo sobre as outras (acoplamento baixo).

```
Coesão alta        cada componente com um papel: domínio, busca, conciliação
Acoplamento baixo  dependências apontam para dentro, por portas e contratos
```

- O campo `Componente` do backlog nomeia essas fronteiras
- Uma história que atravessa muitos componentes é sinal de acoplamento alto — e de uma fatia mal cortada

> No projeto de exemplo (MUSI), a ferramenta `arch-go` e o compilador impedem o domínio de conhecer HTTP, e os testes de contrato impedem as três implementações de divergir.

Pair programming e propriedade coletiva

Duas pessoas, um problema: revisão contínua, conhecimento espalhado, menos código que só uma pessoa entende.

No MUSI	Evidência
Par	`Co-authored-by:` no commit
Revisão	PR aprovado por outro integrante (está na DoD)
Propriedade coletiva	Qualquer um mexe em qualquer módulo

> Não precisa parear o tempo todo. Precisa que nenhum trecho crítico tenha só um dono.

Integração contínua — a prática, não só a ferramenta

Integrar é juntar seu trabalho ao dos outros, várias vezes ao dia, com o build verde a cada vez.

```
commit pequeno ⟶ push ⟶ CI roda ⟶ verde
     (várias vezes ao dia, não um massivo na véspera)
```

> No MUSI, `.github/workflows/ci.yml` roda um job por componente; a DoD exige CI verde para o item sair de "Em revisão". A regra de arquitetura do domínio Go é verificada ali, pelo `arch-go`.

Exemplos no projeto de referência (MUSI)

Prática	Onde ver
TDD	testes que carregam `contratos/exemplos/` nas três linguagens
Integração contínua	`.github/workflows/ci.yml`
Regra de arquitetura	`services/arch-go.yml`, verificado no CI
Design registrado	`docs/decisoes/` (ADRs)
Definição de Pronto e rituais	`processo/definicao-de-pronto.md`, `processo/rituais.md`
Backlog e quadro	Project "MUSI" no GitHub · `processo/backlog.md`

> Copiar a estrutura do `processo/` do MUSI é um bom ponto de partida para o acordo de vocês.

As tarefas da Sprint 0

O enunciado de cada entrega está em `docs/SPRINT-0-TAREFAS.md` — uma tarefa por cartão, com *pronto quando* e o peso na rubrica.

Ajuste recente	Detalhe
Backlog: mínimo 5 histórias (antes 10)	≥ 3 estimadas, todas priorizadas
`docs/proposta.md`: até 5 páginas (antes 3)	as 7 seções do guia

> Criem uma issue por tarefa e coloquem na coluna Sprint Backlog. O guia completo, com templates, continua em `docs/SPRINT-0.md`.

Próximos passos

Até a entrega (11/09)

- Backlog refinado no GitHub Projects: ≥ 5 histórias, ≥ 3 estimadas
- `docs/proposta.md` com visão, MVP e o acordo de processo
- Definição de Pronto escrita — comece pela do MUSI e adapte
- Vídeo de 5 minutos, todos falam

> 09/09 é encontro online, no horário da aula, para dúvidas sobre a proposta e o acordo de processo.

Referências da semana

Histórias, fatiamento e estimativa

- Rubin — *Essential Scrum*, cap. 6 (livro completo no SIGAA)

Extreme Programming

- Beck & Andres — *Extreme Programming Explained*, 2ª ed.
- Beck — *Test-Driven Development: By Example*
- Fowler — *Refactoring* · `martinfowler.com`

Livro-texto aberto

- Valente — *Engenharia de Software Moderna* · `engsoftmoderna.info`

Disciplina e projeto de exemplo

- `github.com/fmarquesfilho/processos-2026-2`
- MUSI · `github.com/fmarquesfilho/musi` — pastas `processo/` e `docs/decisoes/`